Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Systems@ETH Zürich

Cloud Computing Architecture

Semester project report

Group 087
Nikita Shubin - 25-738-634
Author 2 - Legi-NR
Author 3 - Legi-NR

Systems Group
Department of Computer Science
ETH Zurich
May 8, 2026

Instructions

- **Please do not modify the template!** Except for adding your solutions in the labeled places, and inputting your group number, names and legi-NR on the title page.

  Divergence from the template can lead to subtraction of points on graded parts of the project.

- Parts 1 and 2 of the project are **ungraded**. They will help you analyze the behavior of the applications you will have to run on parts 3 and 4 (graded), for which you will have to design a scheduling policy **based on the information** you will gather on **parts 1 and 2**.

- Be conservative with your credits! Parts 3 and 4 might need extra credits for experimentation. Parts 1 and 2 should cost you approximately **15 USD**.

- **Use of AI Tools**: The use of AI tools must adhere to ETH regulations. **You take responsibility for the content you submit**. You must disclose any use of GenAI and other AI tools in your work and avoid sharing copyrighted, private, or confidential information with commercial AI platforms. Failure to comply with these guidelines may result in disciplinary action.

Part 1

Using the instructions provided in the project description, run memcached alone (i.e., no interference), and with each iBench source of interference (cpu, l1d, l1i, l2, llc, membw). For Part 1, you must use the following `mcperf` command, which varies the target QPS from 5000 to 80000 in increments of 5000 (and has a warm-up time of 2 seconds with the addition of `-w 2`):

```
$ ./mcperf -s MEMCACHED_IP --loadonly
$ ./mcperf -s MEMCACHED_IP -a INTERNAL_AGENT_IP  \
           --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -w 2 -t 5 \
           --scan 5000:80000:5000
```

Repeat the run for each of the 7 configurations (without interference, and the 6 interference types) **at least 3 times** (3 should be sufficient in this case), and collect the performance measurements (i.e. the `client-measure` VM output).
Reminder: after you have collected all the measurements, make sure you **delete your cluster**. Otherwise, you will easily use up the cloud credits. See the project description for instructions how to make sure your cluster is deleted.

(a) Plot a line graph with the following stipulations:

- Queries per second (QPS) on the x-axis (the x-axis should range from 0 to 80K). (**note:** the actual achieved QPS, not the target QPS)
- 95th percentile latency on the y-axis (the y-axis should range from 0 to 6 ms).
- Label your axes.
- 7 lines, one for each configuration. Add a legend.
- State how many runs you averaged across and include error bars at each point in both dimensions.

Answer:

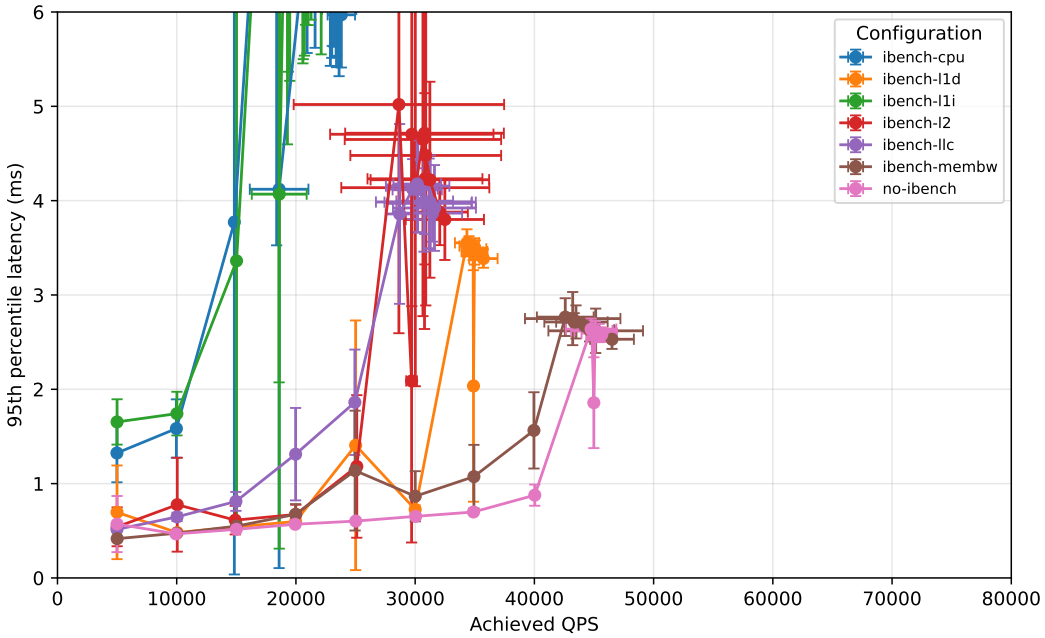

Figure 1: 95th percentile latency as a function of achieved QPS. Each point is averaged across 3 runs. Error bars show one standard deviation across runs in both achieved QPS and 95th percentile latency.

(b) How is the tail latency and saturation point (the "knee in the curve") of memcached affected by each type of interference? What is your reasoning for these effects? Briefly describe your hypothesis.

Answer:

- `ibench-cpu`: CPU interference has one of the strongest negative effects. Even at low load, the 95th percentile latency is already much higher than the no-interference baseline: around 1.3-1.6 ms at 5-10K QPS, compared with about 0.5 ms for the baseline. The curve rises sharply around 15-20K QPS and then the achieved throughput stops increasing significantly, saturating around 23-24K QPS.

  Explanation: memcached is highly sensitive to CPU contention: when the interfering benchmark consumes CPU execution resources, memcached has less compute time available to process requests, causing queueing delays and much worse tail latency.

- `ibench-l1d`: L1 data-cache interference has a noticeable but smaller effect than CPU or L1 instruction-cache interference. The curve remains relatively close to the baseline up to around 20-30K QPS, although there is some variability around 25K QPS. After about 30-35K QPS, the p95 latency increases to around 3.4-3.5 ms and achieved QPS plateaus around 35K QPS.

  Explanation: memcached depends on fast access to request metadata, keys, and internal data structures, so L1 data-cache pollution increases memory access latency. However, because the L1 data cache is private to a core and the interference may not always collide perfectly with memcached's working set, the effect is weaker than CPU, L2, or LLC contention.

- `ibench-l1i`: L1 instruction-cache interference has a very strong effect, similar to or worse than CPU interference. The p95 latency is already around 1.6-1.7 ms at only 5-10K QPS,

and the curve rises sharply by around 15-20K QPS. After that point, achieved QPS remains around 20-22K QPS while p95 latency often exceeds 6 ms.

Explanation: the interfering workload causes instruction-cache and front-end contention, forcing memcached to repeatedly reload parts of its instruction working set. Since memcached must execute many small request-processing operations quickly, extra instruction-fetch stalls can strongly increase tail latency.

- `ibench-l2`: L2 interference causes a strong reduction in the saturation point. Latency is still moderate at low QPS, but it begins to increase around 25K QPS and reaches about 4-5 ms once achieved QPS is around 30K. The system then saturates around 30-32K QPS, well below the baseline saturation point of around 45K QPS.

  Explanation: memcached relies significantly on cache locality beyond L1. When the L2 cache is polluted by the interfering benchmark, more accesses must go to slower cache levels or memory, increasing request service time and creating queueing at lower throughput.

- `ibench-llc`: LLC interference also strongly hurts memcached. The latency starts rising earlier than in the baseline: p95 is already about 1.3 ms at 20K QPS and about 1.9 ms at 25K QPS. The knee appears around 25-30K QPS, after which achieved QPS stays around 30-32K and p95 latency remains around 4 ms.

  Explanation: memcached has a relatively large memory-resident working set, so the last-level cache is important for avoiding expensive memory accesses. LLC contention reduces the effective cache capacity available to memcached, causing more cache misses and therefore higher tail latency.

- `ibench-membw`: Memory-bandwidth interference increases tail latency, but in your measurements it does not reduce the saturation point as much as CPU, L1I, L2, or LLC interference. The curve stays fairly close to the baseline until roughly 35-40K QPS, then p95 latency rises to about 2.5-2.8 ms while achieved QPS levels off around 43-46K QPS. This suggests that memory bandwidth does affect memcached, especially near saturation, but it is not the primary bottleneck at low and medium QPS in this setup.

  Explanation: memcached benefits from cache hits for much of its request path; therefore, bandwidth contention becomes most visible only when request rate is high enough that cache misses and memory traffic become frequent.

(c) Processor affinity:

- Explain the use of the `taskset` command in the container commands for memcached and iBench in the provided scripts.

  Answer: `taskset` pins a process to specific CPU cores. In the provided scripts, it is used to control whether memcached and iBench share the same core or only share higher-level resources. This makes the interference experiment more precise: each iBench benchmark stresses a particular hardware resource.

- Why do we run some of the iBench benchmarks on the same core as memcached and others on a different core? Give an explanation for each iBench benchmark.

  Answer: The scripts place CPU, L1D, L1I, and L2 interference on the same core as memcached because these resources are core-local. LLC and memory-bandwidth interference run on another core because those resources are shared across cores.

(d) Assuming a service level objective (SLO) for memcached of up to 2 ms 95th percentile latency at 35K QPS, which iBench source of interference can safely be collocated with memcached without violating this SLO? Briefly explain your reasoning.

Answer:

(e) In the lectures you have seen queueing theory.

- Is the project experiment above (ignoring client measure) an open system or a closed system? Explain why.
  Answer:

- What is the number of clients in the system? How did you find this number?
  Answer:

- Sketch a diagram of the queueing system modeling the experiment above. Give a brief explanation of the sketch.
  Answer:

- Provide an expression for the average response time of this system. Explain each term in this expression and match it to the parameters of the project experiment.
  Answer:

(f) Below we ask you to do a theoretical cost analysis of the above experiments you ran and compare with the actual cost of your experiments, for which you will **need to track the total runtime** of the experiments, from deploying the cluster to its deletion. To find the actual cost, you can look at the **detailed billing report** after it is available on your Google Cloud project.

For the theoretical one, it would be helpful to check the official pricing calculator (link) or given individual pricing list (link). You also need to see the `machineType` and `subnets` parameters of each VM configuration by examining `part1.yaml`.

Note you can see the actual configuration details (e.g. disk size and type) of each VM configuration by looking at each individual VM instance after you deployed your cluster in `Compute Engine > VM instances > <your vm name>`.

- What is the cost per hour (in USD) for running each VM, according to the VM pricing?
  Answer:

- How much time (approximately) did your experiments take?
  Answer:

- What is the expected (theoretical) total cost and what was the actual cost of running those experiments?
  Answer:

- Does the expected cost match the actual cost? (i.e. equivalent ± 0.01 if rounded up to 2 decimals). If they do not match, explain how you could achieve a more accurate estimate of the cost.

  Answer:

Part 2

1. **Interference behavior**

   (a) Fill in the following table with the normalized execution time of each batch job with each source of interference. The execution time should be normalized to the job's execution time with no interference. Round the normalized execution time to 2 decimal places. Color-code each cell in the table as follows: **green** if the normalized execution time is less than or equal to 1.3, **orange** if the normalized execution time is over 1.3 and less or equal to 2, and **red** if the normalized execution time is greater than 2. The coloring of the first three cells is given as an example of how to use the cell coloring command. **Do not change the structure of the table. Only input the values inside the cells and color the cells properly.**

Workload	none	cpu	l1d	l1i	l2	llc	memBW
barnes	1.00	1.47	1.73	2.03	1.78	2.50	1.74
blackscholes	1.00	1.29	1.37	1.65	1.45	1.43	1.32
canneal	1.00	1.18	1.50	1.39	1.45	1.86	1.33
freqmine	1.00	1.87	1.65	1.93	1.25	1.87	1.54
radix	1.00	1.13	1.34	1.16	1.28	1.67	1.30
streamcluster	1.00	1.22	1.24	1.33	1.34	1.62	1.15
vips	1.00	1.56	1.76	1.70	1.79	1.82	1.61

   (b) Explain what the interference profile table tells you about the resource requirements for each job. Give your reasoning behind these resource requirements based on the functionality of each job.

   Answer:

   - `barnes:` Strong sensitivity to cache-related interference, especially LLC and L1I (up to 2.50x).
     Explanation: Barnes (N-body style computation) repeatedly traverses hierarchical data structures; performance depends heavily on cache locality and instruction/data reuse. LLC pressure causes many expensive misses.

   - `blackscholes:` Moderate sensitivity overall; strongest degradation under L1I/L2/LLC (about 1.4–1.65x).
     Explanation: Blackscholes is mostly compute-oriented and fairly regular, but still suffers when instruction front-end and cache hierarchy are perturbed.

   - `canneal:` Moderate sensitivity, with LLC being the worst (1.86x), CPU impact relatively small (1.18x).
     Explanation: Canneal performs many irregular memory accesses during optimization/annealing steps; it is more cache-capacity sensitive than pure-core-execution limited.

   - `freqmine:` Very sensitive to CPU, L1I, and LLC contention (about 1.87–1.93x), but less sensitive to L2 (1.25x).
     Explanation: Frequent pattern mining has branch-heavy and memory-intensive phases; contention in execution/front-end plus shared LLC strongly hurts runtime.

- `radix`: Most robust workload in this set (mostly 1.13–1.34, except LLC 1.67).
  Explanation: Radix sort has relatively regular access and good parallel structure; it still depends on shared last-level cache, but tolerates other interference types better.

- `streamcluster`: Mild-to-moderate sensitivity; LLC dominates (1.62x), memBW impact is low (1.15x).
  Explanation: Streamcluster repeatedly updates cluster assignments/centers; this creates cache pressure, especially at shared LLC, while raw memory bandwidth is not the first bottleneck.

- `vips`: Broadly sensitive to almost all interference types (1.56–1.82x).
  Explanation: Image-processing pipelines have mixed compute and memory behavior with sizable working sets; they are affected by multiple resource-contention modes, especially shared caches.

(c) Which jobs (if any) seem like good candidates to collocate with memcached from Part 1, without violating the SLO of 2 ms P95 latency at 40K QPS? Explain why.

Answer: At 40K QPS, the Part 1 results show that memcached meets the SLO (p95 $\leq$ 2 ms) only under relatively mild contention (notably `membw`; p95 is still below 2 ms), while CPU/L1I/L2/LLC contention clearly violates the SLO.

Therefore, there are no "perfectly safe" batch jobs at full aggressiveness, because all PARSEC jobs show at least moderate cache sensitivity and can generate cache pressure. The most promising candidates are `radix` and `streamcluster`, since they are comparatively less sensitive/less disruptive across most interference types, and are easier to throttle (e.g., fewer threads or lower CPU share). In contrast, `barnes`, `freqmine`, and `vips` are risky due to strong cache-related contention.

2. **Parallel behavior**

(a) Plot a line graph with speedup as the y-axis (normalized time to the single thread config, Time_1 / Time_n) vs. number of threads on the x-axis (1, 2, 4 and 8 threads - see the project description for more details). Pay attention to the readability of your graph.

Answer:

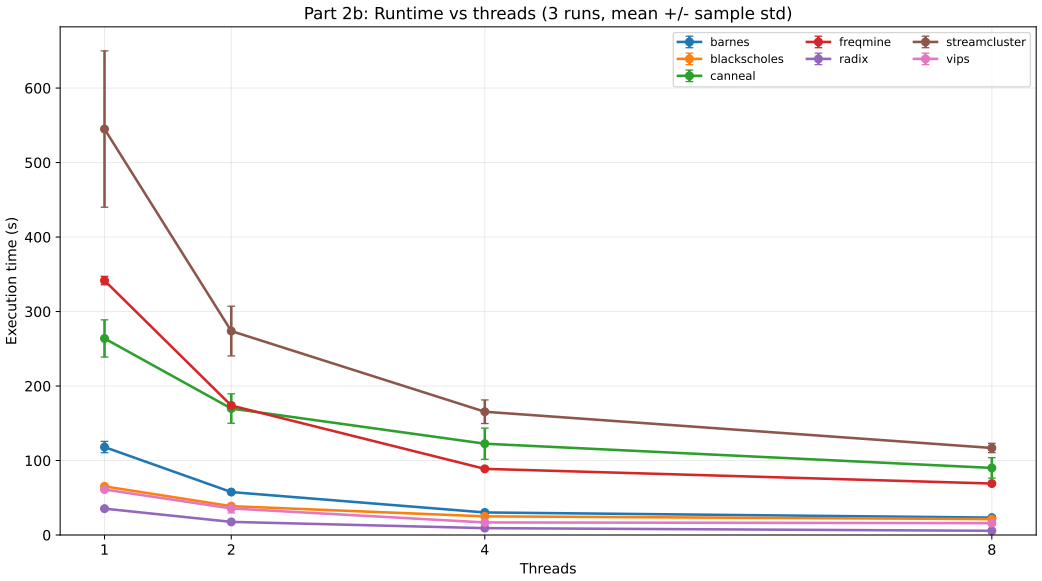

Figure 2: Speedup vs. number of threads for all workloads in Part 2b. Speedup is defined as T_1/T_n. Each point is averaged across 3 runs; error bars show one standard deviation.

(b) Briefly discuss the scalability of each job: e.g., linear/sub-linear/super-linear. Do the applications gain significant speedup with the increased number of threads? Explain what you consider to be "significant".

Coloring rule used in the table. To avoid inconsistent visual interpretation across different thread counts, colors are assigned by *parallel efficiency*, not by raw speedup:

$$S(t) = \frac{T_1}{T_t}, \qquad E(t) = \frac{S(t)}{t}.$$

We use:

- **Green**: $E(t) \geq 0.75$ (high scalability),
- **YellowOrange**: $0.55 \leq E(t) < 0.75$ (moderate but still useful scalability),
- no highlight: $E(t) < 0.55$ (weak scalability at that thread count).

This explains, for example, why `freqmine` at $t = 8$ with $S(8) = 4.95$ is YellowOrange:

$$E(8) = \frac{4.95}{8} = 0.619,$$

which is moderate (not high) efficiency.

Benchmark	t=1	t=2	t=4	t=8
barnes	1.00	2.05	3.90	5.05
blackscholes	1.00	1.69	2.61	3.06
canneal	1.00	1.56	2.17	2.95
freqmine	1.00	1.97	3.85	4.95
radix	1.00	2.01	3.84	6.28
streamcluster	1.00	1.98	3.28	4.65
vips	1.00	1.75	3.63	3.91

Answer: Answer:

- `barnes`: **Scalability type:** strong sub-linear. Speedup grows almost proportionally up to 4 threads ($S(2) = 2.05$, $S(4) = 3.90$), which indicates a large parallel fraction. At 8 threads, growth continues but with diminishing returns ($S(8) = 5.05$, ideal would be 8), suggesting increasing overheads (thread coordination, memory contention, and non-parallel sections). **Conclusion:** substantial and practically meaningful gains across all thread counts, especially up to $t = 4$.

- `blackscholes`: **Scalability type:** consistently sub-linear, moderate. The speedup improves from 1.69 (2 threads) to 2.61 (4 threads), but the gain from 4 to 8 threads is small ($2.61 \rightarrow 3.06$). This flattening indicates that additional threads mostly amplify overhead rather than useful parallel work. **Conclusion:** multithreading helps, but scaling quality is moderate and weak at higher thread counts.

- `canneal`: **Scalability type:** weak sub-linear. It has the lowest scaling profile overall ($S(8) = 2.95$), and efficiency drops early already beyond 2–4 threads. This behavior is typical when synchronization, irregular memory access, or serial phases dominate execution. **Conclusion:** only limited benefit from increasing threads; returns diminish quickly.

- `freqmine`: **Scalability type:** strong sub-linear. It scales very well to 4 threads ($S(4) = 3.85$, near ideal 4), then still improves at 8 threads ($S(8) = 4.95$) but with clear diminishing returns. Importantly, the absolute speedup at 8 threads is high, yet efficiency is moderate ($E(8) = 4.95/8 = 0.62$), so it is not in the top scalability tier. **Conclusion:** strong practical speedup, especially up to 4 threads; moderate efficiency at 8 threads.

- `radix`: **Scalability type:** best in the set (near-linear early, strong overall). It is almost ideal at 2 and 4 threads (2.01, 3.84), and remains the strongest at 8 threads (6.28). Compared to all other jobs, it preserves the highest efficiency at larger thread counts, indicating relatively low parallel overheads. **Conclusion:** highly scalable; thread increase gives clearly significant and robust speedup.

- `streamcluster`: **Scalability type:** good sub-linear. The growth is strong and steady (1.98, 3.28, 4.65), showing real parallel benefit. However, variability is higher than for some other benchmarks, so realized gains may fluctuate between runs. **Conclusion:** significant average speedup with acceptable, but not perfect, scalability stability.

- `vips`: **Scalability type:** two-phase sub-linear (good early, weak late). Scaling is excellent up to 4 threads ($S(4) = 3.63$), then nearly saturates by 8 threads ($S(8) = 3.91$). The small $4 \rightarrow 8$ increment suggests that bottlenecks outside parallel compute dominate at higher concurrency. **Conclusion:** strong benefit up to 4 threads; limited additional value from moving to 8 threads.

What is considered "significant" speedup? To stay consistent with the table

coloring, we define significance using 8-thread parallel efficiency:

$$E(8) = \frac{S(8)}{8}.$$

We classify benchmarks as:

- **Strongly significant:** $E(8) \geq 0.70$,
- **Moderately significant:** $0.50 \leq E(8) < 0.70$,
- **Weak:** $E(8) < 0.50$.

Using this criterion:

- **Strongly significant:** `radix` ($E(8) = 0.785$),
- **Moderately significant:** `barnes` (0.631), `freqmine` (0.619), `streamcluster` (0.581),
- **Weak:** `blackscholes` (0.382), `canneal` (0.369), `vips` (0.489).

Part 3 [68 points]

i. **[34 points]** Describe and analyze your hand-crafted policy.

A. **[17 points]** With your scheduling policy, run the entire workflow **3 separate times**. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached, running with a steady client load of 30K QPS. For each batch application, compute the mean and standard deviation of the execution time [1] across three runs. Also, compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Finally, compute the SLO violation ratio for memcached for the three runs; the number of data points with 95th percentile latency > 1ms, as a fraction of the total number of data points. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

job name	mean time [s]	std [s]
barnes		
blackscholes		
canneal		
freqmine		
radix		
streamcluster		
vips		
total time		

Answer:

Create 3 bar plots (one for each run) of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Using the augmented version of mcperf, you get two additional columns in the output: `ts_start` and `ts_end`. Use them to determine the width of each bar in the bar plot, while the height should represent the p95 latency. Align the x axis so that $x = 0$ coincides with the starting time of the first container. For each core in each machine show what job it is running using the colors proposed in this template (you can find them in `main.tex`). For example, draw a bar in the color of vips for core x from when vips is started to when it ends if vips ran on core x.
Plots:

B. **[17 points]** Describe and justify the "optimal" scheduling policy you have designed.

- Which node does memcached run on? Why?
  Answer:

- Which node does each of the 7 batch jobs run on? Why?

[1] Here, you should only consider the runtime, excluding time spans during which the container is paused.

Answer:
- barnes:
- blackscholes:
- canneal:
- freqmine:
- radix:
- streamcluster:
- vips:

- Which jobs run concurrently / are colocated? Why?
  Answer:

- In which order did you run the 7 batch jobs? Why?
  Order (to be sorted): barnes, blackscholes, canneal, freqmine, radix, streamcluster, vips
  Why:

- How many threads have you used for each of the 7 batch jobs? Why?
  Answer:
  - barnes:
  - blackscholes:
  - canneal:
  - freqmine:
  - radix:
  - streamcluster:
  - vips:

- Which files did you modify or add and in what way? Which Kubernetes features did you use?
  Answer:

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above). Describe how your policy (and its performance) compares to another policy that you experimented with (in particular to the second-best policy that you designed).
  Answer:

ii. **[34 points]** Describe and analyze the AI-generated policy. [TODO: Xiaozhe]

A. **[17 points] Report the performance**:

- Report the mean time of each batch job, as well as the makespan of all jobs for the AI-generated policy. Also, report the SLO violation ratio for memcached for the AI-generated policy. *Note: If the LLM failed to produce a valid solution after 3+ attempts, provide the error logs and your best-performing manual tweak for partial credit.*

job name	mean time [s] (Baseline)	mean time [s] (AI)
barnes		
blackscholes		
canneal		
freqmine		
radix		
streamcluster		
vips		
total time		

- Create 3 bar plots (one for each run), for the final AI-generated policy, of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Use the same method as described in the previous question to create the bar plots.
  Answer:
- Create two line plots (one for the p95 latency and one for SLO violations) showing policy performance over iterations.
  Answer:

B. **[17 points]** Analysis of the Evolutionary Process: Describe the process of how you used OpenEvolve to design the AI-generated policy. For example, you can include the following details in your answer (You are not limited to these questions. You can include any other details you find relevant and interesting). Your response will be graded based on the thoroughness of the exploration and clarity in explanation:

- How many iterations did you run through OpenEvolve with the LLMs to get to the final policy? Which LLM(s) did you use?
- How do you measure success? For example, how do you design the 'combined_score' or the success metric, given that there are two objectives (makespan and SLO violations)?
- Describe the initial policy you fed to the LLM(s) and why you chose this as the initial policy.
- Describe how the policy changes during the process, and how it impacted the performance of the policy (e.g., tail latency; SLO violations).
- Explain why did this AI-generated policy perform better (or worse) than the baseline?
- Identify one "hallucination" or logical flaw the model produced during the process and how you identified it.

Answer:

Please attach your modified/added YAML files, run scripts, OpenEvolve scripts, experiment outputs and the report as a zip file. You can find more detailed instructions about the submission in the project description file.

Important: The search space of all possible policies is exponential and you do not have enough credits to run all of them. We do not ask you to find the policy that minimizes the total running time, but rather to

design a policy that has a reasonable running time, does not violate the SLO, and takes into account the characteristics of the first two parts of the project.

Part 4 [75 points]

i. **[19 points]** Use the following `mcperf` command to vary QPS from 5K to 125K in order to answer the following questions:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP  \
           --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 2 \
           --scan 5000:125000:10000
```

a) [8 points] How does memcached performance vary with the number of threads (T) and number of cores (C) allocated to the job? In a single graph, plot the 95th percentile latency (y-axis) vs. achieved QPS (x-axis) of memcached (running alone, with no other jobs collocated on the server) for the following configurations (one line each):

- Memcached with T=1 thread, C=1 core
- Memcached with T=1 thread, C=2 cores
- Memcached with T=1 thread, C=3 cores
- Memcached with T=2 threads, C=1 core
- Memcached with T=2 threads, C=2 cores
- Memcached with T=2 threads, C=3 cores
- Memcached with T=3 thread, C=1 cores
- Memcached with T=3 thread, C=2 cores
- Memcached with T=3 threads, C=3 cores

Label the axes in your plot. State how many runs you averaged across (we recommend three runs) and include error bars. The readability of your plot will be part of your grade.

Plots:

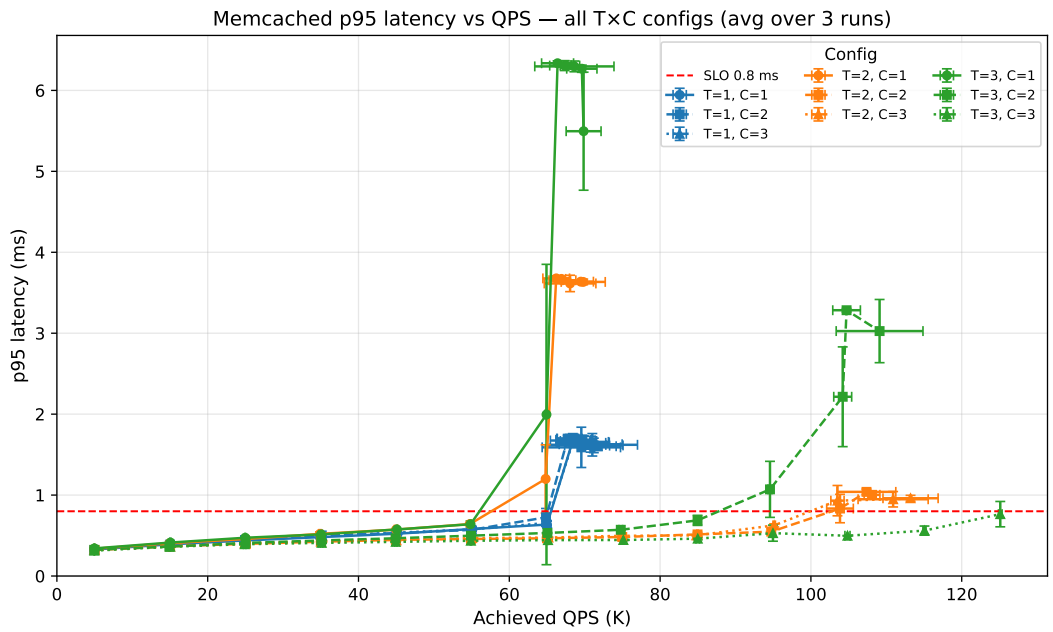

Figure 3: 95th percentile latency vs QPS. For all $T \times C$ configs. Each point is averaged across 3 runs. Error bars show one standard deviation across runs in both achieved QPS and 95th percentile latency.

What do you conclude from the results in your plot? Summarize in 2-3 brief sentences how memcached performance varies with the number of threads and cores and how this relates to the target QPS.

Summary:

- With $T = 1$ and any number of cores, the server saturates at $\sim$ 70k QPS with latency starting to violate the SLO after $\sim$ 65k QPS.
- One CPU and and any number of threads does not exceed $\sim$ 70k QPS. With one CPU, the higher the number of threads – the poorer the latency for similar achieved QPS is – $T = 3$: $\sim$ 6ms; $T = 2$: $\sim$ 3.5ms; $T = 1$: $\sim$ 1.5ms
- $T = 2$ and $C = 2, 3$ achieve almost 100k QPS before violating SLO. These runs achieve almost 118k QPS with around 1ms latency.
- Only $T = 3; C = 3$ achieves 125k QPS without breaking the SLO.

b) [2 points] To support the highest load in the trace (around 125K QPS) without violating the 0.8ms latency SLO, how many memcached threads (T) and CPU cores (C) will you need?

Answer: Only $T = 3; C = 3$ achieves 125k QPS without breaking the SLO.

c) [1 point] Assume you can change the number of cores allocated to memcached dynamically as the QPS varies from 5K to 125K, but the number of threads is fixed when you launch the memcached job. How many memcached threads (T) do you

propose to use to guarantee the 0.8ms 95th percentile latency SLO while the load varies between 5K to 125K QPS?
Answer: We need $T \geq 2$ so it can use whatever cores given. $T = 3$ shows to hurt actual performance with less then 3 cores, while $T = 2$ shows similar performance with $C = 2, 3$

d) [8 points] Run memcached with the number of threads T that you proposed in (c) and measure performance with $C = 1$, $C = 2$ and $C = 3$. Use the aforementioned `mcperf` command to sweep QPS from 5K to 125K.
Measure the CPU utilization on the memcached server at each load time step. Plot the performance of memcached using 1-core ($C = 1$), 2 cores ($C = 2$) and 3 ($C = 3$) in **three separate graphs**, for $C = 1$, $C = 2$ and $C = 3$, respectively. In each graph, plot achieved QPS on the x-axis, ranging from 0 to 125K. In each graph, use two y-axes. Plot the 95th percentile latency on the left y-axis. Draw a dotted horizontal line at the 0.8ms latency SLO. Plot the CPU utilization (ranging from 0% to 100% for $C = 1$, 200% for $C = 2$ and 300% for $C = 3$) on the right y-axis. For simplicity, we do not require error bars for these plots.
Plots:

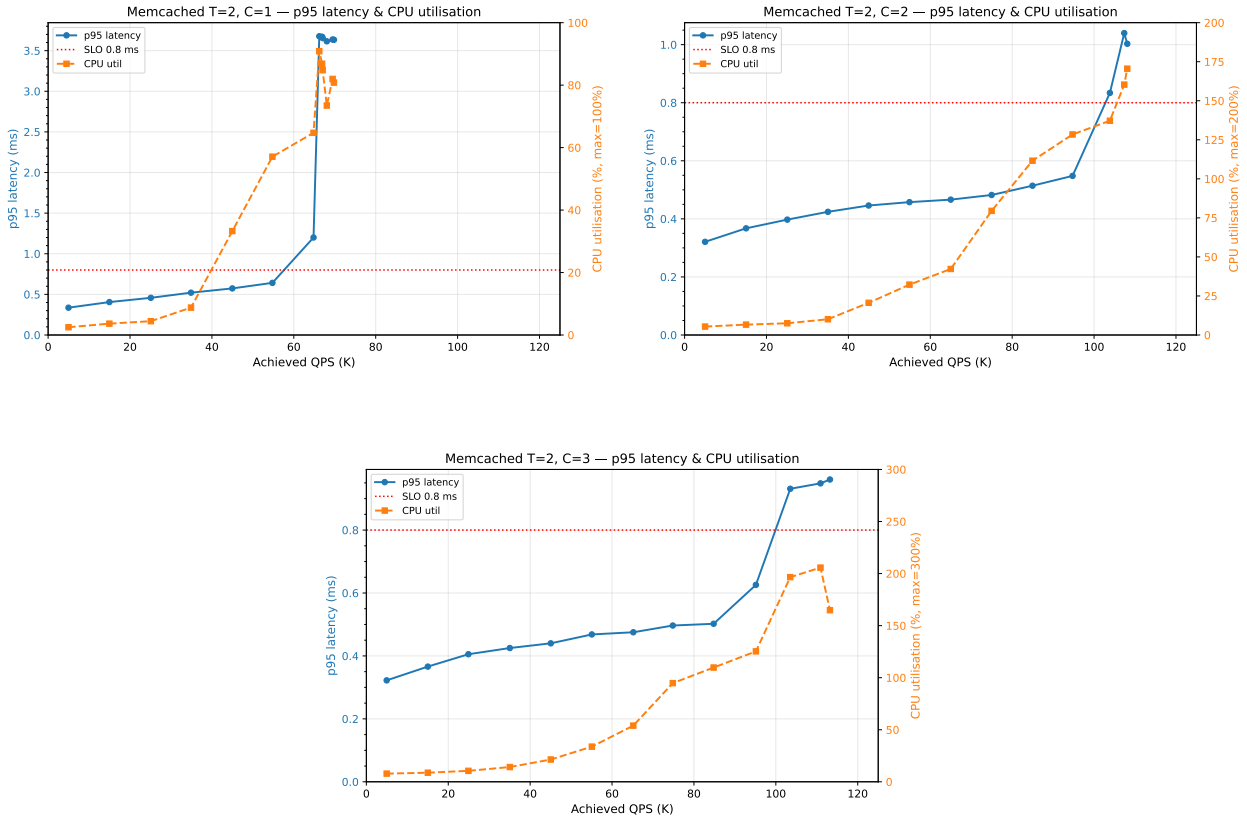

Figure 4: Latency and CPU utilization with various memcached configurations.

ii. **[17 points]** You are now given a dynamic load trace for memcached, which varies QPS randomly between 5K and 110K in 15 second time intervals. Use the following

command to run this trace:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
           --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
           --qps_interval 15 --qps_min 5000 --qps_max 110000
```

Note that you can also specify a random seed in this command using the `--qps_seed` flag.
For this and the next questions, feel free to reduce the mcperf measurement duration (`-t` parameter, now fixed to 30 minutes) as long as you have at the end at least 1 minute of memcached running alone.

Design and implement a controller to schedule memcached and the benchmarks (batch jobs) on the 4-core VM. The goal of your scheduling policy is to successfully complete all batch jobs as soon as possible without violating the 0.8ms 95th percentile latency for memcached. **Your controller should not assume prior knowledge of the dynamic load trace. You should design your policy to work well regardless of the random seed.** The batch jobs need to use the native dataset, i.e., provide the option `-i native` when running them. Also make sure to check that all the batch jobs complete successfully and do not crash. Note that batch jobs may fail if given insufficient resources.

Describe how you designed and implemented your scheduling policy. Include the source code of your controller in the zip file you submit.

- Brief overview of the scheduling policy (max 10 lines):
  Answer:

- How do you decide how many cores to dynamically assign to memcached? Why?
  Answer:

- How do you decide how many cores to assign each batch job? Why?
  Answer:
  - barnes:
  - blackscholes:
  - canneal:
  - freqmine:
  - radix:

  - streamcluster:
  - vips:

- How many threads do you use for each of the batch job? Why?
  Answer:
  - barnes:
  - blackscholes:
  - canneal:
  - freqmine:
  - radix:
  - streamcluster:
  - vips:

- Which jobs run concurrently / are collocated and on which cores? Why?
  Answer:

- In which order did you run the batch jobs? Why?
  Order (to be sorted): barnes, blackscholes, canneal, freqmine, radix, streamcluster, vips
  Why:

- How does your policy differ from the policy in Part 3? Why?
  Answer:

- How did you implement your policy? e.g., docker cpu-set updates, taskset updates for memcached, pausing/unpausing containers, etc.
  Answer:

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above). Describe how your policy (and its performance) compares to another policy that you experimented with (in particular to the second-best policy that you designed).
  Answer:

iii. **[23 points]** Run the following `mcperf` memcached dynamic load trace:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
           --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
           --qps_interval 15 --qps_min 5000 --qps_max 110000 \
           --qps_seed 2345
```

Measure memcached and batch job performance when using your scheduling policy to launch workloads and dynamically adjust container resource allocations. Run this workflow 3 separate times. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached. For each batch application, compute the mean and standard deviation of the execution time [2] across three runs. Compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Also, compute the SLO violation ratio for memcached for each of the three runs; the number of data points with 95th percentile latency > 0.8ms, as a fraction of the total number of datapoints. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

job name	mean time [s]	std [s]
barnes		
blackscholes		
canneal		
freqmine		
radix		
streamcluster		
vips		
total time		

Answer:

Include three plots – one for each of the three runs – with the following information: The x-axis should be the time from execution start to end. Use three subplots that share the same x-axis. The first should clearly indicate for each core what job it is running at all time (e.g. using colored boxes). Use the colors proposed in this template (you can find them in `main.text`). If you pause/unpause jobs as part of your scheduling policy this should also be clearly visible. The second subplot should include two y-axis where the left one shows the current QPS and the right one should include the achieved p95 latency. Finally, the third subplot should plot

[2] Here, you should only consider the runtime, excluding time spans during which the container is paused.

the cpu utilization of each core separately. The readability of your plot will be part of awarded points.
Plots:

iv. **[16 points]** Repeat Part 4 Question 3 with a modified `mcperf` dynamic load trace with a 5 second time interval (`qps_interval`) instead of 15 second time interval. Use the following command:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
           --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
           --qps_interval 5 --qps_min 5000 --qps_max 110000 \
           --qps_seed 2345
```

You do not need to include the plots or table from Question 3 for the 5-second interval. Instead, summarize in 2-3 sentences how your policy performs with the smaller time interval (i.e., higher load variability) compared to the original load trace in Question 3.
Summary:

What is the SLO violation ratio for memcached (i.e., the number of datapoints with 95th percentile latency $>$ 0.8ms, as a fraction of the total number of datapoints) with the 5-second time interval trace? The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.
Answer:

What is the smallest `qps_interval` you can use in the load trace that allows your controller to respond fast enough to keep the memcached SLO violation ratio under 3%?
Answer:

What is the reasoning behind this specific value? Explain which features of your controller affect the smallest `qps_interval` interval you proposed.
Answer:

Use this `qps_interval` in the command above and collect results for three runs and fill out the table below.

job name	mean time [s]	std [s]
barnes		
blackscholes		
canneal		
freqmine		
radix		
streamcluster		
vips		
total time		

Plots:

Use of AI Tools: Did you use any AI tools for this project? If so, disclose them here and explain what you used the tool(s) for and why. Note that the use of AI tools is NOT needed and NOT expected for the project.